Examination 2025-08-19

Introduction to Parallel Programming (5 hours)

Please make sure to write your exam code on each page you hand in. When you are finished, please put the pages containing your solutions together in an order that corresponds to the order of the questions. You can take with you this exam if you want.
This is a **closed book** examination but you are allowed to have with you an **A4 sheet of prepared hand-written notes**. The exam contains **40** points in total and their distribution between sub-questions is clearly shown. To get a final grade of **5** you must score at least **34**. To get a grade of **4** you must score at least **28** and to pass the exam you must score at least **19**. Note that you will get **credit only for answers that are correct (or a very serious attempt) and clear**. All answers should preferably be **in English**; if you are uncomfortable with English you might of course use Swedish. Whenever in *real doubt* for what a particular question might mean, **state your assumptions clearly**.

DON'T PANIC!

1. **General Questions (6 pts total; 2 each).**

   (a) What types of mutexes are supported by Pthreads? What are their main differences?
   (b) What are read-write locks and what are their main advantages compared to mutex locks?
   (c) Explain what the terms blocking and non-blocking synchronization mean and how these two synchronization mechanisms compare against each other.

2. **Amdahl's Law (6 pts total; 3 each).**

   (a) You have a choice between buying one uniprocessor that executes five zillion instructions per second, or a ten-processor multiprocessor where each processor executes one zillion instructions per second. Using Amdahl's Law, explain how you would decide which to buy for a particular application. (A *zillion* is a fictitious, indefinitely large number.)
   (b) A friend of yours who works for a company developing parallel software claims that, despite Amdahl's law, in practice many parallel programs obtain excellent speedups. Can you think of reasons for this to hold or is your friend exaggerating?

3. **Concurrency Errors (6 pts total; 3 each).** Give an example of a linked list implementation (using C++/Pthreads pseudocode) and a sequence of memory accesses to the linked list in which the following pairs of operations can potentially result in problems:

   (a) Two `deletes` executed concurrenty.
   (b) An `insert` and a `delete` executed concurrenty.

4. **Pthread Programming (4 pts total; 2 each).** A program spawns T threads to calculate, numerically, the area under a function from 0 to 1: $\int_0^1 f(x)dx$. The precision is controlled by a parameter N that sets how many trapezes each thread computes.

   Example: for $T = 4$ threads, each thread computes an equally large section of the integral:

   - Thread 0 computes: $\int_0^{\frac{1}{4}} f(x)dx$.
   - Thread 1 computes: $\int_{\frac{1}{4}}^{\frac{2}{4}} f(x)dx$.
   - Thread 2 computes: $\int_{\frac{2}{4}}^{\frac{3}{4}} f(x)dx$.
   - Thread 3 computes: $\int_{\frac{3}{4}}^{1} f(x)dx$.

   Assume that the program has been implemented well: no major performance bugs exist.

   You are executing this program on a computer that gives this output when you run the `lscpu` command (some details omitted):

```
$ lscpu
Architecture:          x86_64
CPU(s):                4
Thread(s) per core:    1
Core(s) per socket:    4
Socket(s):             1
CPU MHz:               1600.000
CPU max MHz:           2267.0000
CPU min MHz:           1600.0000
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              8192K
```

   (a) What scaling behaviour (not measuring setup-times) would you expect to see when increasing the number of threads?

   (b) What maximum speed up would you expect, and why?

---

5. **Concurrent Data Structures (4 pts).** *Immutable data structures* are data structures that consist of memory that is never changed; instead of changing a pre-existing object in the data structure, a new object is allocated instead. To give a simple example, a *mutable linked list* (not thread safe) might have an operation `prepend` that inserts a new integer at the beginning:

```
void prepend(int value) {
  // create a new node that has .next point the current first
  Node new_first = new Node(value, this.first);
  // update the list:
  this.first = new_first;
}
```

   Similarly, an *immutable linked list* could have a `prepend` operation that returns a new, slightly modified list, instead of changing the pre-existing list:

```
List prepend(int value) {
  // create a new node that has .next point at the current first
  Node new_first = new Node(value, this.first);
  // return a new list that points at new_first
  return new List(new_first);
}
```

In the context of *concurrency*, what are the main advantages that immutable data structures (not only lists) have over mutable ones? Are there any disadvantages? Briefly explain your answers.

---

6. **OpenMP Programming (6 pts total).** A programmer has parallelized a `foo` function using OpenMP pragmas as follows:

```
int foo(int n, int *a, int *b, int *c, int *d) {
  double tmp, total = 0;
  #pragma omp parallel
  {
    #pragma omp sections
    {
      #pragma omp section
      {
        #pragma omp for
        for (int i = 0; i < n; i++) {
          a[b[i]] += b[i];
          total += b[i];
        }
      }
      #pragma omp section
      {
        #pragma omp for
        for (int i = 0; i < n; i++) {
          tmp = c[i];
          c[i] = d[i];
          d[i] = tmp;
          total += c[i];
       }
       #pragma omp for
       for (int i = 0; i < n; i++) {
         total += d[i];
       }
      }
    }
  }
  return total;
}
```

If you think there are any issues (i.e., errors, omissions in the pragmas, or pragmas which are missing) with this code, explain why they are issues and suggest a (good) way to fix them.

---

7. **MPI Programming (4 pts).** In lecture 11, we examined various forms of MPI communication. Suppose `comm_sz` $= 8$ and the vector `x` $= (0, 1, 2, \ldots 15)$ has been distributed among the processes using a block distribution. Draw a diagram illustrating the steps in a butterfly implementation of `allgather` of `x`.

---

8. **Parallel Performance (4 pts; 2 pts each).** Your friends are devastated: their program does not scale! Luckily, you are around. Your friends' program runs four threads, each thread executes the function `doWork`. Here is how it looks like:

```
float partialResults[4];

void doWork(int threadID) {
  for (int i = 0; i < 10000000; ++i) {
    partialResults[threadID] += computeSegment(threadID, i);
  }
}

int main(...) {
  ... // initialize partialResults, spawn worker threads, then join them
  float sum = 0.0;
  for (int i = 0; i < 4; ++i) { sum += partialResults[i]; }
  printf("sum = %f\n", sum);
}
```

The program has very short sequential parts, so your friends expected their program to scale linearly. However, the runtime barely improves for two threads, and it gets even worse for more threads than two. The function `computeSegment` is not our concern, it does only boring, but tedious mathematical computations.

(a) What is the problem, in your opinion?

(b) How would you fix the problem? Rewrite the relevant parts of the code.

Good luck !